Rust, Strings, and Text

CS/COE 1622

Jarrett Billingsley

Class Announcements

- I'm assuming, for this lecture, that you really did follow along with chapters 1-3 of the Rust book
- also, if you didn't see the announcement yesterday...
  - go look :^)
- the examples from today are in the examples repo
  - see the materials page!

Vectors

Vectors!

- Rust's **`Vec<T>`** can work like a Java **`T[]`** or **`ArrayList<T>.`**

```
let v = vec![1, 2, 3, 4, 5];
```

this creates a new, *immutable* (unchangeable) Vec.

```
let mut v = vec![1, 2, 3, 4, 5];
```

this one can be modified, appended, etc.

for the most part, they work like Java arrays/ArrayLists:

```
let x = v[3];     // indexing
let l = v.len(); // length
v.push(6);        // appends a 6
```

Iteration

- Rust has a system similar to Java's **`Iterable`**/**`Iterator`** interfaces.
  - **`.iter()`** is like Java's **`.iterator()`** method.
- the **`for-in`** loop iterates over iterators.

```
for i in 0 .. 10 {}     // i = 0, 1, 2, …, 8, 9
for i in 1 ..= 10 {}    // i = 1, 2, …, 9, 10

let v = vec![1, 2, 3];
for val in v.iter() {} // iterate over values in v
for val in &v {}       // same as above, but shorter
for (i, val) in v.iter().enumerate() {}
```

the last one shows calling a method on an iterator object to get a new iterator – this one gives the index *and* value.

an Extremely Common Pattern: Mapping

- you have an array A.
- you want to perform some transformation on each item in A.
- you want to put the results of those transformations into an array B.

```
int[] B = new int[A.length()];
for(int i = 0; i < A.length(); i++) {
     B[i] = A[i] * 2;
}
```

this operation is called **mapping,** and it's so, so common.
in Rust, it looks like this:

```
let B = A.iter().map(|x| x * 2).collect();
```

this is a **function literal,** or
anonymous function, or **"lambda."**

Algebraic Data Types and Structs

time check: ≤ 35 min

Not the same as ADTs (*abstract* data types)

- **primitive** types contain a single value, e.g. **`i32`**, **`f32`**, **`bool`**, **`char`**...
- **algebraic** types let us **combine** types into larger ones.

Product Types bundle values "side-by-side".

```
class Point {
  int x;
  int y;
}
```

Java classes are product types. each Point can hold any *combination* of two integers.

Sum Types hold one of several *choices* or **variants.**

```
boolean b;
b = true;
b = false;
// that's it.
```

Java's **`boolean`** can **only** be one of two possibilities, but that's *built into the language* for you.

Rust's product types: Structs

- **structs** are the main kind. (there are also "tuples.")

```
struct Point {
  x: i32,
  y: i32,
}
```

they're like Java classes, but no **inheritance,** no **interfaces,** no **methods,** just **fields.**

```
let p = Point { x: 10, y: 20 };
```

you construct a struct by listing the values for each field.

```
println!("{}, {}", p.x, p.y);
```

and you access fields with `.` just like in Java, C, etc.

other than privacy (put `pub` on a field to make it public), *that's it.*

"Constructors" and "methods"

- Rust **does not have classes** and is **not** an OOP language.
- but it does let us add methods to **any** type, with **`impl`.**

```
impl Point {
    fn new(x: i32, y: i32) -> Point {
        Point { x, y }
    }

    fn flip_x(&self) -> Point {
        Point { x: -self.x, y: self.y }
    }
}
```

a Rust convention is to have a **constructor** function named **`new`**.

methods are made by writing **`&self`** as the first argument.

now we can access these as **`Point::new()`** and **`p.flip_x()`**.

A side note - :: vs .

- in Java, you use `.` to access all "things inside other things".
  - e.g. **`java.util.Arrays`, `Integer.parseInt()`**
- in Rust, the `.` operator is only used **on objects.**
  - when you see `.` it always means "get a field from a struct" or "call a method on some value."
- for anything else, you use `::`
  - e.g. **`std::cmp::Ordering`, `Point::new()`**
  - I like to think of `::` as being like the (back)slashes in file paths
    - **`C:\Windows\system32`**
    - **`/bin/sh`**

Flavors of 'self'

- inside an **impl** there are a few kinds of functions:

```
impl MyStruct {
    fn new() -> MyStruct
```

with **no** `self` argument, it's an **associated function.** constructors are the most common example.

```
    fn method(&self)
```

with a `&self`, it's a **method,** but cannot change any fields in the object it's called on.

```
    fn mutator(&mut self)
```

with a `&mut self`, it's a method that *can* change the object, but can only be called on `mut` variables.

Sum Types

time check: ≤ 65 min

A motivating example

- the classic Animal class hierarchy might look like this in Java:

```
abstract class Animal {
  abstract void speak();
}
class Cat extends Animal {
  void speak() { println("meow!"); }
}
class Dog extends Animal {
  void speak() { println("woof!"); }
}
class Camel extends Animal {
  void speak() { println("ghhghhg!"); }
}
```

an **Animal** variable would be able to hold an instance of any of these.

common fields and methods go in **Animal;** **unique** things go in the subclasses.

The equivalent (?) Rust

- when you want to have a **choice** of types, you use an **enum.**

```
enum Animal {
  Cat, Dog, Camel
}

impl Animal {
  fn speak(&self) {
    use Animal::*;
    match self {
      Cat   => println!("meow!"),
      Dog   => println!("woof!"),
      Camel => println!("ghhghhg!"),
    }
  }
}
```

what's happening here is not *really* like the Java code, though.

it's much more like a **boolean:** a **fixed set** of possible values, and an **Animal** variable can only take on those values.

but unlike Java, *we get to define our own types like this* in Rust.

Commonalities and differences

- in OOP, we can reuse code/fields by putting them in the base class…
  - and we can specialize code/fields in the subclasses.

```
abstract class Animal {
  float weight;
  Animal(float w) { weight = w; }
  void printWeight() { println(weight); }
}
class Cat extends Animal {
  String pattern;
  Cat(float w, String p) {
    super(w); pattern = p;
  }
  void printPattern() { println(pattern); }
}
```

here, **weight** is common to all **Animals**, but **pattern** is unique to **Cats.**

When a struct and an enum love each other very much

- in Rust, this can be represented with this pattern that has no name:

```
struct Animal {
  weight: f32,
  kind:   AnimalKind,
}

enum AnimalKind {
  Cat   { pattern: String },
  Dog   { loudness: i32 },
  Camel { num_humps: i32 },
}
```

common fields appear in this **Animal** struct. but it also has a **kind** field which is an enum.

unique fields appear in the **enum** variants.

this pattern is going to show up *over and over!*

it definitely looks different than the Java code, but the same concepts are coming into play.

Looking inside enums

- there are two ways to see what's inside the enum:

```
// "if k is an AnimalKind::Cat,
// extract its data and assign it to 'pattern'."
if let AnimalKind::Cat { pattern } = &self.kind {
  println!("cat with pattern: '{}'", pattern);
}
```

```
match &self.kind {  .. means "throw away" or "I don't care"
  AnimalKind::Cat{..} => println!("meow!"),
  AnimalKind::Dog{..} => println!("woof!"),
  _                   => println!("???"),
}
```

NullPointerException? Never heard of it

- **there is no such thing as null in Rust!** 
- well, there's something *like* it, but **it's opt-in, not forced on you.**
- it's an enum called **Option.**

```
let nullable_ints: Vec<Option<i32>> = vec![
  Some(1), Some(45), None, Some(7) ];

for i in nullable_ints {
  if let Some(v) = i { println!("{}", v); }
}
```

Option is so widely used that it's built into the compiler.
you don't have to write **Option::None**, for instance.

Strings and Unicode

time check: ≤ 90 min

Soooo easy

- **lexing** is the process of splitting the source text into **tokens.**
- that means we'll have to deal with **strings** as the input to our lexer.

if you're a native English speaker, you're probably used to thinking of strings as working like this:

0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*
`h`	`e`	`l`	`l`	`o`	`!`	`\n`	`:`	`)`	`\n`
104	101	108	108	111	33	10	58	41	10

a string is an **array of characters**, where each character is encoded as a small integer. easy, right?

Those dang human languages

- the reality is: **it's way more complicated than that.**
- see **`StringWeirdness.java`** for some… string weirdness.

```
String s = "tést";
String t = "tést";
System.out.println(s.length());  // 4
System.out.println(s.charAt(1)); // é
System.out.println(t.length());  // 5??
System.out.println(t.charAt(1)); // e??
```

welcome to the wonderful world of Unicode!!!

Unicode

- **Unicode** is The String Encoding that the whole world* uses now.
- it attempts to map *every written character in every human language* to a number, its **codepoint.**
- but it's more complicated than that, because human writing systems.

é = [U+00E9]

é = [U+0065, U+0301]

e + ◌́

this is a **combining mark:** it **modifies** the previous glyph.

UTF-8? UTF-16? UTF-32? BE? LE?

- each codepoint is *conceptually* a **21-bit number.**
- Unicode text is **encoded** into one of a few **transformation formats.**
- the encodings of [`U+0074`, `U+00E9`, `U+0073`, `U+0074`]:

UTF-32: each codepoint is a **32-bit int.** simple, but wastes space.

LE	74	00	00	00	e9	00	00	00	73	00	00	00	74	00	00	00
BE	00	00	00	74	00	00	00	e9	00	00	00	73	00	00	00	74

UTF-16: each is a 16-bit int... **unless** it's in a certain range, in which case it's *two* 16-bit ints.

LE	74	00	e9	00	73	00	74	00
BE	00	74	00	e9	00	73	00	74

UTF-8: ASCII characters are **1 byte**; all others are 2, 3, or 4 bytes long.

74	c3	a9	73	74

UTF-8 has become the dominant encoding.

Such sights to show you

- **Rust uses UTF-8 strings** (and so do many other languages).
- Unicode and UTF-8 mean you have to **abandon many of your assumptions about how strings work.**

what is this string's **length?**

t	あ	゜	s	t				
74	e3	81	82	cc	8a	73	74	
in bytes?	0	1	2	3	4	5	6	7	len=8
in codepoints?	0	1			2		3	4	len=5
in "*grapheme clusters?*"	0	1					2	3	len=4

also, because each codepoint is a **different number of bytes,**
indexing by codepoints is **not necessarily O(1)!**

Oh god

- what about getting a **substring**?
- what about **reversing** strings?
- what about **toUpper/toLower**?
- if we **loop over a string,** do we see bytes, codepoints, or clusters?
- should the "e with acute" precomposed character **compare equal** to the "e followed by combining acute" character pair?
- what about **collating** ("alphabetizing")?
- what if there are **multiple combining marks** on a character? does the *order* of those marks affect its display or meaning?



it's not Unicode's fault, it's ours.

What about Java?

- Java adopted Unicode early! ...maybe a little *too* early.
- in Java, **`String`** is **UTF-16**, and **`char`** is **16 bits** (2 bytes).
  - at the time Java was created, *that's what Unicode was*.
- **but Unicode was extended to 21 bits *after* Java adopted it.** so...
  - **`.charAt()` *is* O(1), nice!**
  - ...but **you might only get *half a character??***
- see, codepoints above U+FFFF are encoded as *two* Java **`char`**s
  - e.g. many emoji are in this range
- so, "proper" codepoint indexing is **O(n)** in Java too
  - there is a **`.codePointAt()`** method to help, at least
  - and a **`.codePoints()`** iterator

Living in a post-ASCII world

- **do not assume:**
  - ...that your program will only be used by English speakers.
  - ...that your program will only be fed ASCII text.
  - ...that there is one definition of "string length."
  - ...that string indexing/substring is a constant-time operation.
- **do:**
  - **trust** the people who do this stuff for a living.
  - read about how your programming language handles text.
  - write your text processing to be **language-agnostic.**
  - prefer **iterating** over strings instead of **indexing** them, if possible.
  - look into **canonicalization** if you're dealing with thorny issues.
  - accept that some Rust string stuff looks weird/complicated for a *very good reason:* **text is weird/complicated.**